

Core Concepts of the OpenGL ES Rendering Pipeline

A Practical Guide to OpenGL ES, Shaders, and Rendering APIs

Table of Contents

1. Introduction to OpenGL ES
2. GLAD and Android OpenGL ES
3. Understanding Vertex Shaders
4. Understanding Fragment Shaders
5. Understanding of Primitive Assembly and Rasterization.
6. Understanding Vertex and Fragment Shaders Through Triangle example
7. Understanding Common OpenGL APIs
8. Setting Up OpenGL in Visual Studio Desktop
9. Learning OpenGL Through Practical Examples
 1. Drawing a Triangle
 2. Drawing a Single Line
 3. Drawing Parallel Line
 4. Drawing a Square
10. Code Overview

1. Introduction to OpenGL ES

- **OpenGL ES** (Open Graphics Library for Embedded Systems) is an open-standard graphics library designed for embedded and mobile devices.
- It is a graphics API used to render 2D and 3D graphics on Android and other embedded platforms.
- OpenGL ES acts as a bridge between an application and the Graphics Processing Unit (GPU), enabling efficient graphics processing.
- It uses programmable shaders, such as Vertex Shaders and Fragment Shaders, to generate the final image displayed on the screen.
- Android provides a framework and APIs that allow developers to integrate OpenGL ES into their applications.
- OpenGL ES is widely used in applications that require high-performance graphics, such as camera preview rendering, guidance line overlays, gaming etc

2. GLAD, Android OpenGL ES and GLFW

- **GLAD (OpenGL Loader Generator)** is a library that loads OpenGL function pointers at runtime.
- GLAD is commonly used with Desktop OpenGL applications on Windows, Linux, and macOS.
- Android OpenGL ES applications typically do not use GLAD.
- Android provides direct access to OpenGL ES APIs through the Android framework and GLES libraries such as GLES20, GLES30, and GLES32.
- Unlike desktop OpenGL applications, Android does not require a separate OpenGL loader because the OpenGL ES APIs are already exposed through the Android SDK and NDK.
- GLFW (Graphics Library Framework) is an open-source library used for window and OpenGL context creation and Android uses SurfaceView, GLSurfaceView instead of glfw.

3. Understanding Vertex Shaders

- A **Vertex Shader** is a small program written in GLSL (OpenGL Shading Language) that runs on the GPU as part of the OpenGL rendering pipeline.
- Its primary responsibility is to determine the final position of each vertex on the screen.
- The shader source code can be stored in text files (such as .txt, .vert, or .glsl) or embedded within application source files (.c, .cpp, .java, etc.).
- It is executed once for every vertex of a graphical object.
- Ex: If we want to draw a triangle containing 3 vertices, the Vertex Shader is executed 3 times, once for each vertex.
- It can also pass data such as colors, texture(to render camera feed) to the Fragment Shader.

A simple Vertex shader code

Vshader1.txt

```
#version 330 core
layout (location = 0) in vec3 aPos;
void main()
{
    gl_Position = vec4(aPos,1.0);
}
```

Vshader2.txt

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aCol;
out vec3 aColor;
void main()
```

```

{
    gl_Position = vec4(aPos,1.0);
    aColor = aCol;
}

```

#version 330 core : Specifies the GLSL version used by the shader program.

layout (location = 0) : Specifies the attribute index used to bind vertex position data to the shader variable aPos.

in : Declares an input variable.

out : Declares as a output variable

vec4,vec2 : Represents a n-dimensional vector containing n floating-point values

aPos : Stores the vertex position data passed from the application.

aCol : Stores the color data passed from the application.

gl_Position : built-in output variable in the Vertex Shader that stores the final position of a vertex. This value is used by OpenGL to determine where the vertex appears on the screen.

aColor : Passes the color data from the Vertex Shader to the Fragment Shader.

4. Understanding Fragment Shaders

- **A Fragment Shader** is a small program written in GLSL (OpenGL Shading Language) that runs on the GPU as part of the OpenGL rendering pipeline.
- Its primary responsibility is to determine the color and other attributes of each fragment generated during rasterization.
- The shader source code can be stored in text files (such as .txt, .vert, or .glsl) or embedded within application source files (.c, .cpp, .java, etc.).
- It is executed once for each fragment generated by the rasterizer. A fragment is a candidate pixel that may eventually become a pixel on the screen.
- Ex: If we want to draw a triangle containing 100 pixel, the Fragment Shader is executed 100 times, once for each fragment/pixel.
- It can also take data as input from the vertex shader.

A simple Fragment shader code

Fshader1.txt

```
#version 330 core
out vec4 FragColor;
void main()
{
    FragColor= vec4(1.0, 1.0, 1.0, 1.0);
}
```

Fshader2.txt

```
#version 330 core
out vec4 FragColor;
in vec4 vcol;
void main()
{
    FragColor= vcol;
}
```

Here :

vcol : input data from the vertex
shader

FragColor : is a output variable
stores the final color

5. Understanding of Primitive Assembly and Rasterization.

Primitive Assembly : After processing all vertices, OpenGL assembles the vertices into geometric primitives such as points, lines, or triangles according to the specified draw mode.

For example:

GL_POINTS → Points

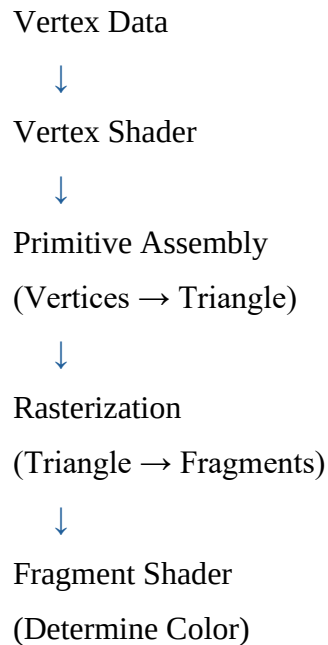
GL_LINES → Lines

GL_TRIANGLES → Triangles

If triangle means provide 3 vertices, OpenGL assembles them into a triangle.

Rasterization : Once a primitive (such as a triangle) is formed, OpenGL converts it into fragments.

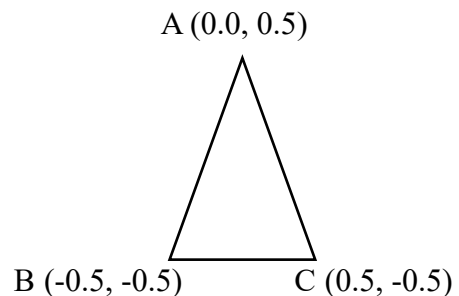
Simple Pipeline Summary



6. Understanding Vertex and Fragment Shaders Through Triangle example.

Ex : **Triangle Vertex data**

```
float vertices[] =  
{  
    0.0f, 0.5f, 0.0f, // A  
    -0.5f, -0.5f, 0.0f, // B  
    0.5f, -0.5f, 0.0f // C  
};
```



First Vertex :

OpenGL reads: Vertex 0 (0.0, 0.5, 0.0)

and sets: `aPos = vec3(0.0, 0.5, 0.0);`

Then executes: `gl_Position = vec4(aPos,1.0);`

Result: `gl_Position = vec4(0.0,0.5,0.0,1.0);`

For next vertices also repeats same thing

Number of Executions : 3 vertices = 3 executions

So internally:

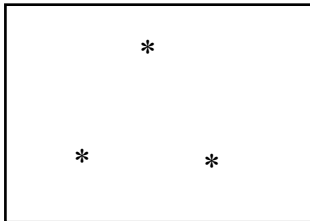
Vertex Shader Call #1

Vertex Shader Call #2

Vertex Shader Call #3

if we have the window of coordinates(x : -1 to +1 y : -1 to +1) OpenGL display them like this on the visible region.

(-1,+1) (+1,+1)



(-1,-1) (+1,-1)

Primitive Assembly : After all vertex shader executions complete:

OpenGL has:

A = (0.0, 0.5)

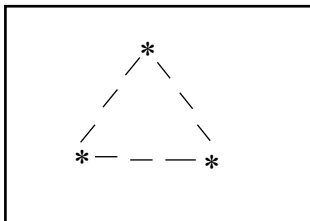
B = (-0.5,-0.5)

C = (0.5,-0.5)

Since we specified: **GL_TRIANGLES**

OpenGL combines every three vertices into a triangle.

(-1,+1) (+1,+1)



(-1,-1) (+1,-1)

This stage is called: Primitive Assembly

Processed V0

Processed V1

Processed V2

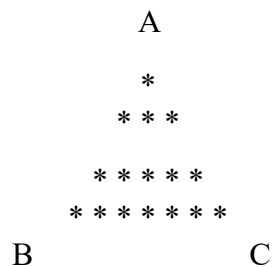


Assemble Triangle <-- only once after the all vertices are done>

Rasterization : Now OpenGL converts the geometric triangle into fragments.

Think: Triangle becomes Thousands of screen pixels

Example:



Every possible pixel inside triangle becomes a fragment.

- Ex: `FragColor = vec4(1.0f, 1.0f, 1.0f, 1.0f);`
- Suppose rasterization generated: 10,000 fragments, then Then fragment shader executes: 10,000 times.
So for each execution: `FragColor = vec4(1.0f, 1.0f, 1.0f, 1.0f);` is called.
- which means: Red = 1, Green = 1, Blue = 1, Alpha = 1
- Result = White Pixel.

Vertex A

Vertex B

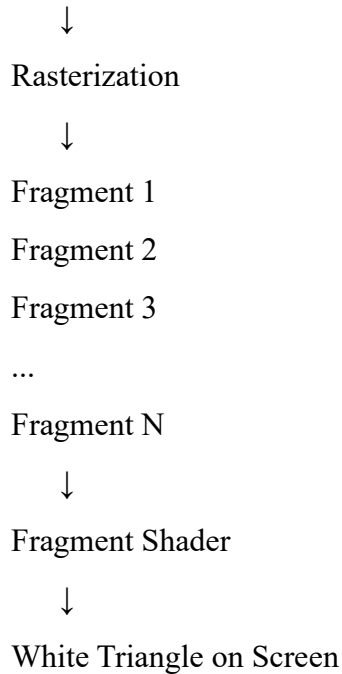
Vertex C



Vertex Shader



Triangle Assembly



7. Understanding Common OpenGL APIs

(A API start with glfw is GLFW API and a API start with gl is OpenGL lib)

- **glfwCreateWindow** : GLFW API used to create a window in a desktop OpenGL application using the GLFW library.

Ex : `window = glfwCreateWindow( 800,600,"Triangle",NULL,NULL);`

800 : Width of the window in pixels

600 : Height of the window in pixels

Triangle : Title displayed in the window's title bar

NULL: Creates the window in windowed mode.

NULL : Tells not sharing any OpenGL context and resources with another window.

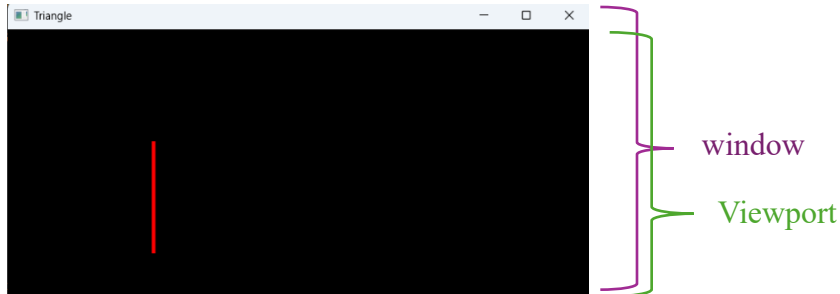
- **glfwMakeContextCurrent** : makes the OpenGL context of the specified window active. Once the context is current, OpenGL API calls are directed to that window, allowing rendering operations such as shader creation, buffer management, and drawing to be performed.

Ex : `glfwMakeContextCurrent(window);`

Window : created window variable

- **glViewport:** defines the rendering area where OpenGL draws the final output within the window.

Ex: `glViewport(0, 0, 800, 600); //rectangular rendering area.`



- **glCreateShader :** Creates a Shader object.

Ex : `vertexShader = glCreateShader(GL_VERTEX_SHADER) ;`

`fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);`

vertexShader: vertex shader object

fragmentShader: fragment shader object

GL_VERTEX_SHADER/ GL_FRAGMENT_SHADER: what type of shader to create.

- **glShaderSource :** Attaches the GLSL source code to a shader object. The shader source is stored in the shader object.

Ex : `glShaderSource(vertexShader, 1,&vertexShaderSource, NULL);`

vertexShader: vertex shader object

vertexShaderSource: containing GLSL shaders source code

NULL: Length of each string. NULL indicates that the shader source code is a null-terminated string.

- **glCompileShader :** Compiles the shader source code and checks for syntax errors.

Ex : `glCompileShader(vertexShader);`

vertexShader: vertex shader object.

- **glCreateProgram :** Creates an empty shader program object

Ex : `shaderProgram = glCreateProgram();`

- glAttachShader:** It attaches a compiled shader object to a shader program object.
 Ex : `glAttachShader( shaderProgram, vertexShader);`
`glAttachShader( shaderProgram, fragmentShader);`
- glLinkProgram :** Links the attached Vertex and Fragment Shaders into a single executable shader program that can run on the GPU.
 Ex: `glLinkProgram( shaderProgram)`
- glGenVertexArrays :** Generates VAO (Vertex Array Object) and stores its unique identifier in VAO.
 Ex: `glGenVertexArrays(1, &VAO);`
1: no of VAO to create object
VAO: Pointer to a variable
- glBindVertexArray :** makes the specified Vertex Array Object (VAO) the currently active VAO.
 Ex : `glBindVertexArray(VAO);`
- glGenBuffers :** creates a buffer object ID that can later be used as a Vertex Buffer Object (VBO) for storing vertex data.
 Ex : `glGenBuffers(1, &VBO);`
1: no of VBO to create object
VBO: Pointer to a variable
- glBindBuffer :** making it the currently active vertex buffer.
 Ex : `glBindBuffer(GL_ARRAY_BUFFER, VBO);`
- glBufferData :** Copies the vertex data from CPU memory (vertices array) into the memory associated with the currently bound VBO (which typically resides in GPU memory).
 Ex : `glBufferData(GL_ARRAY_BUFFER, sizeof(vertices),vertices, GL_STATIC_DRAW);`

GL_ARRAY_BUFFER: Store vertex data

sizeof(vertices) : size of vertices if triangle contain 9 vertices with floating value $9*4=36$ bytes .

vertices : source vertex data.

GL_STATIC_DRAW : data rarely changes.

- **glVertexAttribPointer** : This function defines how OpenGL should read the vertex data from the VBO.

Ex : `glVertexAttribPointer (0,3, GL_FLOAT,GL_FALSE, 3 * sizeof(float), (void*)0);`

0 : Attribute location (aPos)

3 : Read 3 values per vertex (x, y, z).

GL_FLOAT : Data type is float.

GL_FALSE : Do not normalize the values

3 * sizeof(float) : Distance between consecutive vertices (stride)

(void)0*: Start reading from the beginning of the buffer (offset)

- **glEnableVertexAttribArray** : This function enables attribute location 0.

Ex : `glEnableVertexAttribArray(0);`

- **glClearColor** : Choose the background color.

Ex : `glClearColor(0.0f, 0.0f, 0.0f, 1.0f); //Black`

`glClearColor(1.0f, 1.0f, 1.0f, 1.0f); //white`

`glClearColor(1.0f, 0.0f, 0.0f, 1.0f); //Red`

`glClearColor(0.0f, 1.0f, 0.0f, 1.0f); //Green`

`glClearColor(0.0f, 0.0f, 1.0f, 1.0f); //Blue`

- **glClear** : set the color defined by glClearColor.

Ex : `glClear(GL_COLOR_BUFFER_BIT);`

- **glUseProgram** : Activates the specified shader program, making it the current program used for rendering.

Ex : `glUseProgram(shaderProgram);`

- **glBindVertexArray** : binds the specified Vertex Array Object (VAO) and makes it the active VAO. The VAO stores vertex attribute configurations such as attribute locations, data types, strides, offsets, and enabled attributes.

Ex : `glBindVertexArray(VAO);`

- **glLineWidth** : width of the line.

Ex : `glLineWidth(5.0f);`

- **glDrawArrays** :

GPU pipeline begins.

OpenGL reads vertex data from the VBO.

Based on the VAO configuration, position data is passed to 'aPos'.

The Vertex Shader is then executed for each vertex.

After Primitive Assembly, Rasterization, Fragment shader execution happens.

First it will draw the lines on the backscreen and Swap for final screen using

glfwSwapBuffers(window).

ex : `glDrawArrays(GL_LINES,0,2);`

GL_LINES : lines to draw

0 : Starting index of the vertex data. OpenGL begins reading vertices from index **0** in the bound vertex buffer (VBO).

2 : Number of vertices to render.

GL_TRIANGLES : for triangles.

8. Setting Up OpenGL in Visual Studio Desktop

Download GLAD: <https://glad.dav1d.de/>

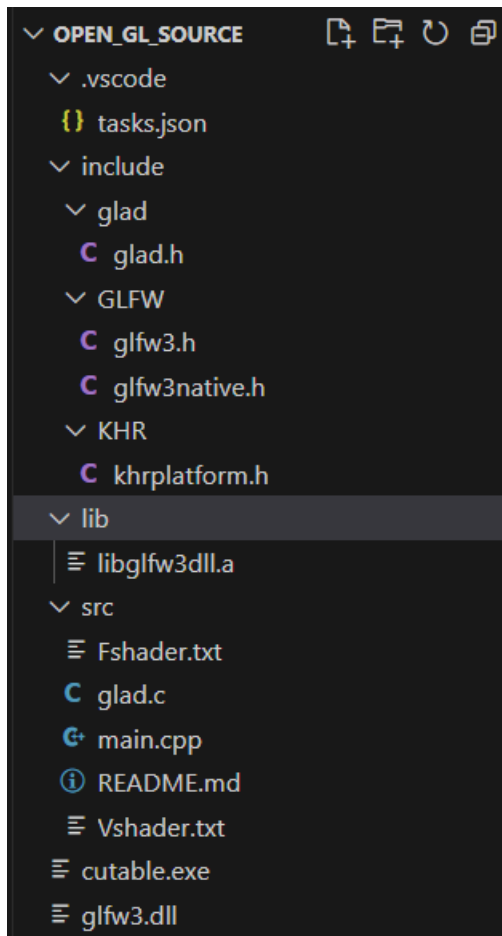
Download GLFW: <https://www.glfw.org/download.html>

Download Link: <https://sourceforge.net/projects/mingw/>

YouTube Link for setting up OpenGL in windows:

<https://www.youtube.com/watch?v=Y4F0tI7WIDs>

Folder Structure



9. Learning OpenGL Through Practical Examples

Sample source code for drawing lines, triangles and square

<https://visteon->

[my.sharepoint.com/personal/khandigu_visteon_com/Documents/Desktop/kavya/OPEN_GL_SOURCE](https://visteon-my.sharepoint.com/personal/khandigu_visteon_com/Documents/Desktop/kavya/OPEN_GL_SOURCE)

Path for shaders and main file :

OPEN_GL_SOURCE/src :

Vshader.txt

Fshader.txt

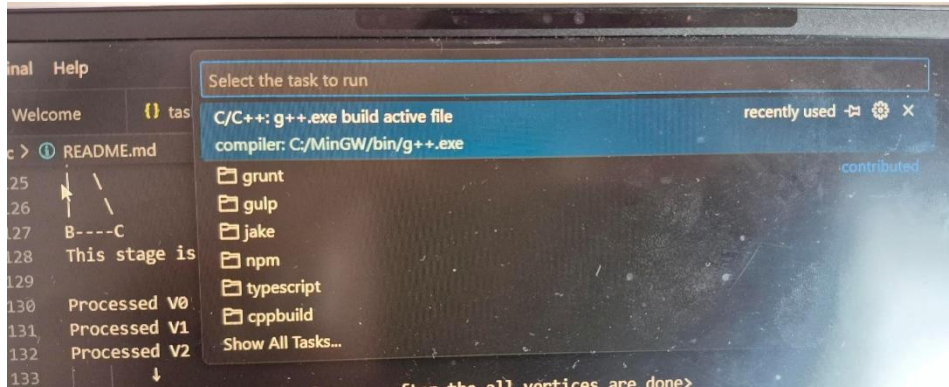
main.cpp

How to compile and run Program :

path OPEN_GL_SOURCE :

and go to **terminal** → choose **Run Task..**

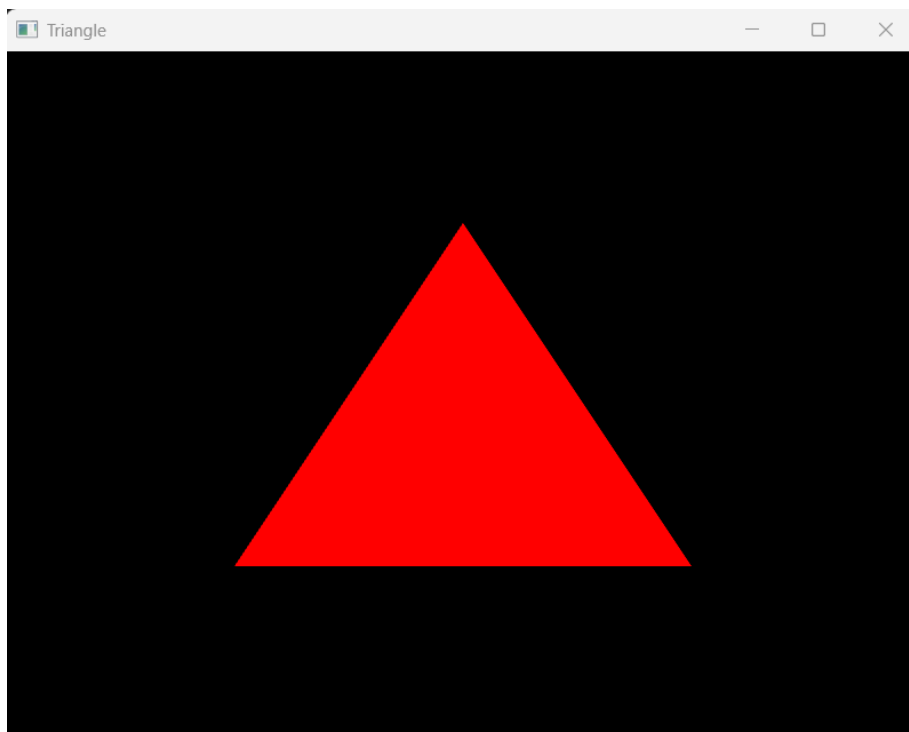
after clicking on Run Task it will show this below click on C/C++ it will compile the whole code.



After successful compilation it will generate one executable called **cutable.exe**

run this in another terminal : **./cutable.exe**

Final output : for Triangle



For Lines

